

COMP64702

Revision Lecture

Week 5 – Week 8

Dr. Jingyuan Sun

How the exam will test this material

Orientation

Think in terms of skills: recognising, calculating, interpreting, and applying.

Select all that apply

Concept distinctions, true/false traps, architecture properties

Numeric calculation

Cosine similarity, TF-IDF, agreement, P/R/F1

Matching / labelling

POS tags, BIO classes, token-level labels

Structure interpretation

Dependency heads, dependents, phrase constituents

Fill-in counts

TP / FP / FN for entities and classes

Scenario choice

RAG vs prompting vs fine-tuning; agents and safety

Reminder:

- *Have a detailed go-through of all previous lecture slides*
- *Do the practice exam.*

High-yield map

The revision is organised around five examinable clusters.

1. Transformers & representations

RNN comparison, positional encoding, attention, embeddings, TF-IDF

2. LLM training & adaptation

Pretraining, instruction tuning, alignment, MoE, reasoning models

3. Prompting & applications

ICL, CoT, RAG, agents, multimodal systems, safety

4. Sequence labelling

POS tagging, dependency parsing, phrase structure, NER/BIO

5. Evaluation

Gold standards, kappa, precision/recall/F1, averaging, entity-level scoring

Transformers vs RNN sequence-to-sequence

Likely format: select the correct conceptual differences.

Aspect	RNN seq2seq	Transformer
Processing	Sequential over tokens	Parallel over input positions
Context	Often bottlenecked by hidden states; attention can help	Dynamic self-attention over all positions
Long-range links	Harder due to recurrence	Direct pairwise attention paths
Order information	Implicit in recurrence	Needs positional information

What to say in an answer

Transformers do not use recurrence; self-attention lets each token build context by attending to other tokens, while positional encodings supply order.

Mini practice

Which property makes Transformer encoders easier to parallelise than RNN encoders?

- A. Causal masking
- B. No recurrence over positions
- C. Learned output labels

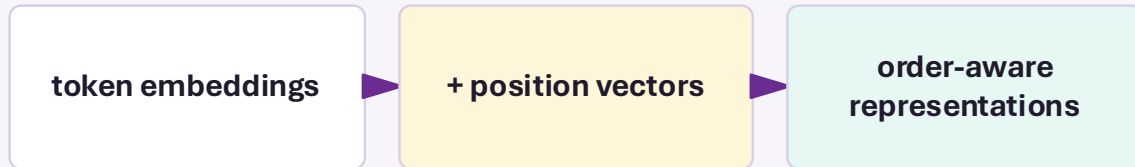
Answer: B

Common trap

Do not claim Transformers have no sequence order; they model order after positional information is added.

Positional encoding

Self-attention alone treats positions symmetrically; order must be injected.



Sinusoidal

- Fixed function of position
- Not learned
- Can extrapolate by formula
- Classic Transformer design

Learned

- Trainable position vectors
- Updated with model weights
- Common in modern systems
- Limited by trained context length

What can be asked?

Identify correct statements about why positional encodings are needed and whether they are learned.

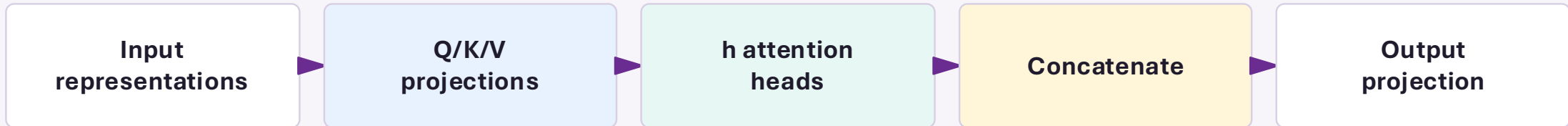
Mini practice

Without positional encodings, why are “dogs chase cats” and “cats chase dogs” hard for plain self-attention?

Answer: It lacks an explicit signal for token order.

Multi-head attention

Likely format: dimensions, projections, and what heads do.



Core idea

Each head attends using its own learned projections. Heads can focus on different representation subspaces or relations.

Dimension rule

A common setup uses $d_{\text{head}} = d_{\text{model}} / h$. The final concatenated output returns to d_{model} before the output projection.

Common trap

Heads are not separate slices of the input sequence. They process the same sequence through different projections.

If $d_{\text{model}} = 768$ and $h = 12$, what is the typical head dimension?
Answer: 64

Transformer architecture families

Know the attention pattern, use case, and examples.

Encoder-only

Examples: BERT, RoBERTa
Bidirectional self-attention
Best for NLU: classification, tagging, extraction
Pretraining often uses MLM-style objectives

BERT ≠ autoregressive

Encoder-decoder

Examples: T5, BART
Encoder represents input
Decoder generates output
Decoder uses self-attention + cross-attention to encoder output

T5 uses cross-attention

Decoder-only

Examples: GPT-style LLMs
Causal masked self-attention
Prompt understanding and generation occur in one left-to-right stream

GPT uses causal masking

Word representations & similarity

Likely format: numeric cosine similarity or conceptual statements about vector methods.

$$\text{cosine}(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| ||\mathbf{B}||)$$

$\mathbf{A}=[1,1,0]$, $\mathbf{B}=[1,0,0]$. Estimate cosine similarity.
Answer: 0.71

Count-based vectors

Term–document or term–term matrices
High-dimensional and sparse
One vector per word type
Can be expensive to store

TF–IDF weighting

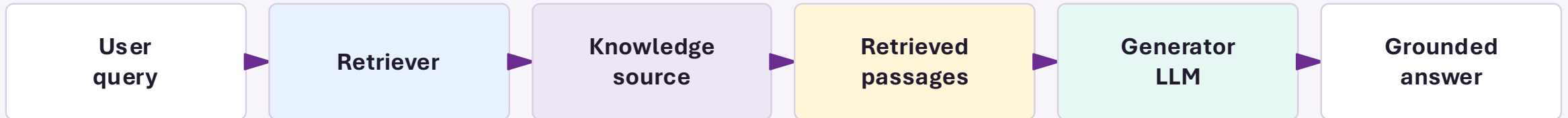
Down-weights very common terms
Emphasises informative terms
Does not magically remove sparsity

Contextual meaning

Static vectors struggle with polysemy
Contextual models can represent a word differently in different sentences

Retrieval-Augmented Generation (RAG)

Know the essential components and when RAG is the right choice.



Essential parts

- External corpus/vector DB
- Retriever
- Generator
- Prompt that combines query + evidence

Use RAG when...

- Facts change often
- Large private documents must be consulted
- You need citation-like grounding
- Fine-tuning is too expensive or inappropriate

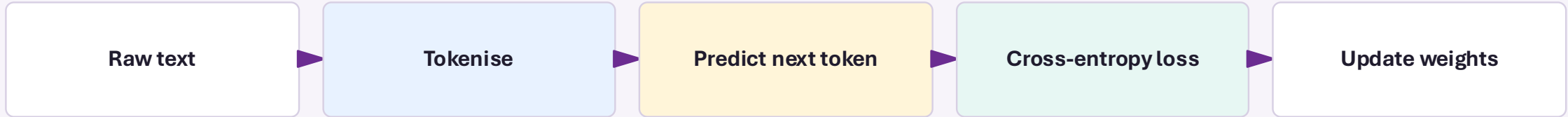
Do not confuse with...

- Pretraining from scratch
- SFT on a handbook only
- A model guessing from parametric memory

A company Q&A bot must answer from a changing policy manual. Which architecture is usually best?
Answer: Instruction-tuned LLM + RAG.

LLM pretraining: objective and limitations

Likely format: distinguish training, inference, and what next-token prediction does not guarantee.



Self-supervised

No manual labels are required. The next token in text supplies the training target.

Training vs inference

Training: compare prediction to ground truth and update weights.
Inference: choose/sample next token and feed it back.

Limitations

Optimises likelihood, not truth. Models may hallucinate, repeat biases, or generate unsafe content.

Autoregressive factorisation: $P(x) = \prod P(x_t | x_{<t})$

Instruction tuning & alignment

Likely format: identify what each stage changes and what it does not guarantee.

Stage	Main purpose
Pretraining	General language modelling from raw text
Instruction tuning / SFT	Follow natural-language instructions using instruction-response pairs
RLHF / preference optimisation	Shift behaviour toward human preferences and safety
DPO / PPO / GRPO	Different optimisation routes for preference alignment

HHH target behaviour

Helpful: answer the task
Honest: say what is known/unknown
Harmless: avoid unsafe or abusive outputs

Common trap

Instruction tuning improves following instructions and zero/few-shot behaviour, but it does not guarantee factual accuracy.

Is instruction tuning inherently the same as RLHF?
Answer: No. It is supervised; RLHF may come afterwards.

MoE and reasoning models

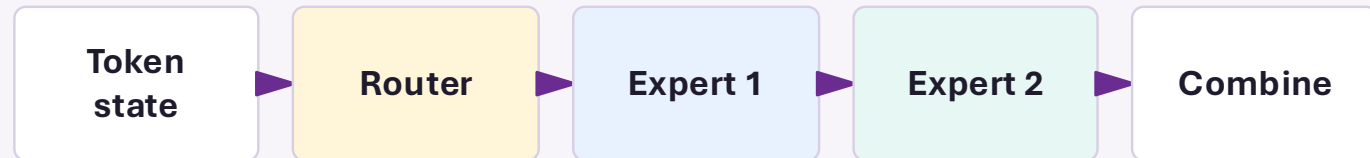
Likely format: conceptual statements about sparsity, routing, and test-time thinking.

Mixture of Experts (MoE)

Several expert networks share a model. A gating/router network selects one or more experts for a token.

Pros: sparse activation, faster pretraining, scaling.

Cons: more complex, memory heavy, fine-tuning challenges.



Reasoning models

They allocate more test-time compute by generating intermediate reasoning before the final response.

Trade-off

Sometimes a smaller model that thinks longer can beat a larger model that answers immediately, but latency/cost increase.

What does “sparse activation” mean in MoE?
Answer: Only a subset of experts runs for each token.

Prompt anatomy and robust prompting

Likely format: identify components of a prompt and best practices.

Instruction

What task should the model do?

Context

Role, constraints, background

Input data

The text, table, image description, or query

Output indicator

Desired format, labels, JSON schema, bullets

Best practices

- Use clear delimiters around user data
- Specify exact output format
- Prefer positive instructions
- Use examples for custom formats
- Check for prompt injection

Mini practice

Improve this prompt:
“Analyse this.”

Make it exam-ready: add instruction, context, input delimiter, output format.

Answer: More specific prompts reduce ambiguity.

In-context learning (ICL)

Know the difference between examples in the prompt and updating model weights.

Mode	What the model receives
Zero-shot	Instruction + target input; no examples
One/few-shot	Instruction + 1–5 demonstrations + target input
Fine-tuning	Training data used to update weights
ICL	No gradient descent; temporary working memory only

Memory view

Parametric memory = model weights learned during training.

Working memory = current prompt/context window.

Mini practice

Does this use ICL?

Translate to Spanish.

English: good morning → Spanish: buenos días

English: thank you → Spanish:

Answer: Yes — it gives a demonstration in the prompt.

Reasoning prompts and prompt security

Likely format: choose the safest or most appropriate prompting strategy.

Chain-of-Thought

Encourages intermediate reasoning before the answer. Useful for multi-step math, logic, and planning. A short trigger can help, but it is not a calculator.

Self-consistency / ToT

Self-consistency samples multiple reasoning paths and majority-votes. Tree of Thoughts explores branches and backtracks.

Prompt injection

Because instructions and data are both text, malicious user content can try to override the task. Separate instructions from data and validate tool use.

For exact arithmetic in an agent, what is safer: pure CoT or a calculator/Python tool?
Answer: Use the external tool and return the observation.

LLM applications and agents

Likely format: map an application scenario to the best adaptation strategy.

Core NLP tasks

Summarisation
Structured extraction / NER
Classification and routing
Question answering

Domain applications

Code: unit tests, bug fixing
Legal/finance: RAG for current facts
Healthcare: strict human review

Agents

Brain: LLM
Memory: context/vector store
Tools: APIs, calculator, browser
Loop: plan → act → observe

Deployment bottlenecks

Inference cost, latency, probabilistic reliability, monitoring, safety, prompt injection, copyright/IP.

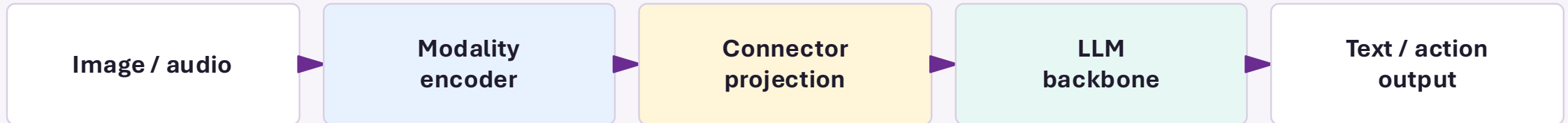
Mini practice

A legal assistant must answer using a changing database of cases.
Prompting only, RAG, or pretraining from scratch?

Answer: RAG, with careful evaluation and human review.

Multimodal LLMs and evaluation

Focus on components, evaluation mismatch, and hallucination.



Architecture terms

Vision encoder or audio encoder
Projection layer / Q-Former
LLM performs complex reasoning and generation

Evaluation issue

Exact string matching can penalise chatty correct answers. Modern MLLM evaluation often uses rubric-based or LLM-as-judge methods.

Hallucination

The model may describe objects or relations absent from the image due to language priors or compressed visual tokens.

Why might a VQA system say “chair” even if no chair is visible near a table?
Answer: Language priors and visual hallucination.

POS tagging

Likely format: match Universal POS tags to tokens under a stated interpretation.

What it is

Assign one POS tag to each token after tokenisation. The same word can have different tags in different contexts.

Tagsets

Penn Treebank: detailed, English-focused.
Universal POS: broader cross-linguistic tags such as PRON, AUX, VERB, NOUN, DET, PUNCT.

How to decide

Use local context and intended meaning. Ask: is the token acting as a noun, verb, auxiliary, determiner, etc.?

Tag the ambiguous word only:

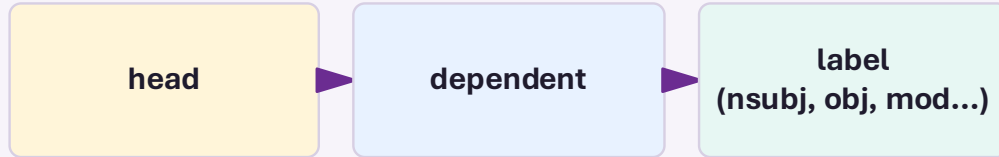
“We watched her duck.”

Meaning A: her pet bird. Meaning B: her quickly lower her head.

Answer: A: NOUN; B: VERB

Dependency parsing

Likely format: identify the root, dependents of a head, or the head of a phrase.



Reading strategy

1. Find the main predicate/root.
2. Identify direct dependents: subject, object, modifiers.
3. For each modifier, ask which word it describes.
4. Use semantics to resolve attachment ambiguity.

Practice sentence

The analyst emailed the report to Maria.

Root: emailed
Subject dependent: analyst
Object dependent: report
PP target/recipient: Maria

Common trap

Do not confuse the preposition itself with the noun it introduces as the semantic head.

Attachment ambiguity

The same words can form different structures depending on meaning.

The reviewer read the article with a chart.

Reading A: event modifier

“with a chart” describes how the reviewer read it.
Attach the PP to the verb/read event.

Reading B: noun modifier

“with a chart” describes which article.
Attach the PP to the noun phrase “article”.

Exam-style action: first paraphrase the intended meaning, then draw/read the dependency or phrase structure accordingly.
Answer: Meaning decides attachment.

Phrase structure parsing

Likely format: identify NPs/VPs/PPs from a phrase structure tree.

Constituents

Phrase structure groups words into nested constituents rather than head-dependent arcs.
Typical nodes: S, NP, VP, PP, AdjP, AdvP.

Noun phrases

Centred on a noun or pronoun. May include determiners, adjectives, possessives, and modifiers.

Not every span

A long sequence is not automatically an NP. Check whether it has a noun/pronoun head and whether it forms a constituent.

Mini practice

In “The student from Leeds solved the puzzle”, name two noun phrases.

Answer: “The student from Leeds”; “Leeds”; “the puzzle”

Named Entity Recognition (NER) and BIO

Likely format: count BIO classes or evaluate exact entity spans.

NER task

Identify spans that mention real-world entities and assign a type: PER, ORG, LOC, DATE, MONEY, etc.

BIO scheme

B-X begins an entity of type X.
I-X continues it.
O means outside any entity.

Class count

For T entity types under BIO:
 $2T + 1$ classes
(B and I for each type, plus O).

If a BIO NER system recognises PER, ORG, LOC, and DATE, how many output classes are needed?
Answer: $2 \times 4 + 1 = 9$

Gold standards, agreement, and Cohen's Kappa

Likely format: compute observed agreement, expected agreement, then kappa.

Why gold standards?

Benchmarks compare system response against reliable reference labels created with annotation guidelines.

Observed agreement

P_a = proportion of items on which annotators agree.
It does not correct for chance agreement.

Expected agreement

P_e = chance agreement estimated from annotator label distributions.

$$\kappa = (P_a - P_e) / (1 - P_e)$$

If $P_a = 0.80$ and $P_e = 0.50$, compute κ .
Answer: 0.60

Precision, recall, F1, and averaging

Likely format: compute scores and explain why accuracy can mislead.

	Predicted +	Predicted -
Gold +	TP	FN
Gold -	FP	TN

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2\text{PR} / (\text{P} + \text{R})$$

Averaging

Micro: pool counts

Macro: average class scores
equally

Weighted: average by class
support

Entity-level NER

Exact span and type must match. A token-level partial overlap is not necessarily a correct entity.

Mini practice

TP=3, FP=1, FN=2. Compute P, R, F1.

Answer: P=.75, R=.60, F1=.667

Final revision checklist

Wrap-up

Use this as a last-pass plan before the exam.

Memorise formulas

Cosine similarity
TF-IDF ingredients
Cohen's Kappa
Precision / Recall / F1
Micro vs macro averaging
BIO class count: $2T + 1$

Practise skills

Select-all conceptual traps
POS tags under a stated meaning
Dependency heads/dependents
Phrase constituents
NER exact-match counting
Architecture selection scenarios

Avoid common mistakes

Treating ICL as weight updates
Calling instruction tuning “guaranteed factuality”
Confusing precision and recall
Counting partial NER spans as correct
Ignoring the stated sentence interpretation

When unsure: define the task, identify the representation, then compute or reason from the definition.